

Microservices concepts & configuration checklist (what you typically need)

This is a practical checklist of the common ****concepts, tools, and configuration**** used in real microservice platforms.

1) Service discovery

****Why****

- Services find each other without hardcoding host/ports.

****Options****

- Kubernetes DNS (often enough on K8s)
- Eureka / Consul (common outside K8s)

****Config****

- Service name (`user-service`, `order-service`)
- Registration + health checks
- Readiness/liveness endpoints

2) API Gateway

****Why****

- Single entry point for routing + security + limits.

****Options****

- Spring Cloud Gateway
- Kong / NGINX / Envoy
- AWS API Gateway

****Config****

- Routes (path/host → upstream service)
- Timeouts (connect/read)
- Retries (careful)
- Rate limiting
- Request size limits
- CORS
- Auth integration (JWT/OAuth2)

3) Centralized configuration

****Why****

- Change config per environment without rebuilding images.

****Options****

- Spring Cloud Config Server (Git-backed)
- Kubernetes ConfigMaps
- AWS SSM Parameter Store

****Config****

- Config per env (`dev`, `qa`, `prod`)
- Refresh strategy (restart vs hot refresh)
- Separation of config vs secrets

4) Secrets management

****Why****

- Protect DB passwords, tokens, signing keys.

****Options****

- AWS Secrets Manager
- HashiCorp Vault
- Kubernetes Secrets (often used, but manage encryption/rotation)

****Config****

- Rotation policies
- Least privilege access
- Audit logging

5) Resilience patterns

****Why****

- Stop cascading failures.

****Patterns/tools****

- Timeouts (mandatory)

- Retries with backoff + jitter (only for safe/idempotent calls)
- Circuit breaker (Resilience4j)
- Bulkheads (isolation)
- Rate limiting

****Config****

- Per-dependency circuit breakers
- Per-client timeout defaults
- Retry policy by endpoint type
- Fallback strategy (degrade vs fail-fast)

6) Distributed tracing

****Why****

- Trace a request across services.

****Options****

- OpenTelemetry + Jaeger/Tempo/Zipkin
- AWS X-Ray

****Config****

- Trace propagation (W3C `traceparent`` or B3)
- Sampling rules (prod typically not 100%)
- Correlation IDs in logs

7) Centralized logging

****Why****

- Search logs across all services.

****Options****

- ELK/EFK
- Loki
- CloudWatch Logs

****Config****

- Structured JSON logs
- `traceId`/requestId``

- Log level per environment
- PII masking

8) Metrics / monitoring / alerting

****Why****

- Know reliability and performance.

****Options****

- Prometheus + Grafana
- Datadog / New Relic
- CloudWatch

****Config****

- SLIs: latency (p95/p99), error rate, throughput, saturation
- SLOs + alerts (error budget)
- Dashboards per service + dependency dashboards

9) Security (authn/authz)

****Why****

- Secure external and internal calls.

****Options****

- OAuth2/OIDC IdPs: Cognito / Okta / Azure AD / Keycloak
- mTLS (often via service mesh)

****Config****

- JWT validation (issuer/JWKS/audience)
- Role/permission model
- Service-to-service auth (JWT, mTLS, SPIFFE)
- Network policies (zero trust)

10) Service-to-service communication

****Types****

- Sync: REST/gRPC

- Async: events/queues

****Config****

- Client libraries (RestClient/WebClient/Feign)
- Timeouts + circuit breaker
- Idempotency keys for creates

11) Messaging / event streaming

****Why****

- Decouple services, enable async workflows.

****Options****

- Kafka
- RabbitMQ
- AWS SQS/SNS

****Config****

- Topics/queues and retention
- Consumer groups
- Retries + DLQ
- Idempotent consumers (at-least-once delivery)

Often required extra service: Schema Registry

If using Kafka events:

- Confluent Schema Registry (or equivalent)
- Define compatibility rules (backward/forward)

12) Data management

****Why****

- Avoid shared database coupling.

****Patterns****

- Database-per-service
- CQRS (sometimes)
- Sagas for distributed workflows

****Config****

- Migrations (Flyway/Liquibase)
- Indexes aligned with queries
- Backup/restore/DR
- Read replicas (if needed)

13) Deployment platform (Kubernetes)

****Required objects****

- Deployment
- Service
- Ingress (optional)

****Config****

- Replicas
- Rolling updates
- Resource requests/limits
- HPA (autoscaling)
- Readiness/liveness probes
- ConfigMaps/Secrets mounts

14) CI/CD

****Why****

- Repeatable builds + safe releases.

****Options****

- GitHub Actions / GitLab CI / Jenkins
- ArgoCD / Flux (GitOps)

****Config****

- Build/test/security scan
- Image build + push
- Deploy strategy: rolling / canary / blue-green
- Rollback strategy

15) Container image registry

****Why****

- Store and pull images.

****Options****

- ECR / GCR / ACR
- Docker Hub

****Config****

- Tagging strategy (semver, git sha)
- Vulnerability scanning

16) Feature flags

****Why****

- Release safely without redeploy.

****Options****

- LaunchDarkly
- Unleash

17) Service mesh (optional but common at scale)

****Why****

- Uniform mTLS, traffic policy, retries, observability.

****Options****

- Istio
- Linkerd

****Config****

- mTLS policies
- traffic shifting
- telemetry

18) Rate limiting and WAF (edge)

****Why****

- Protect against abuse.

****Options****

- Gateway rate limits
- Cloud WAF (AWS WAF)

19) Governance / platform standards

****Why****

- Consistency and speed across teams.

****Examples****

- Shared build conventions (BOMs, parent POMs)
- Templates (service skeleton)
- Standard error response and API guidelines
- Dependency and CVE management process

Summary: “do we need any other tool/service?”

Common additional services beyond just running microservices:

- ****Container registry****
- ****Secrets manager****
- ****Central logging/metrics/tracing backend**** (ELK/Loki + Prometheus + Jaeger/Tempo)
- ****CI/CD****
- ****Message broker/stream**** (Kafka/SQS) and possibly ****schema registry****
- Optionally ****service mesh**** at larger scale

Microservices production setup (User + Order): configurations + implementation examples

This document shows how you typically configure and implement production-grade microservices capabilities using two example services:

- `user-service`
- `order-service`

Assumptions:

- Spring Boot 3.x
- Kubernetes (EKS/GKE/AKS or self-managed)
- OAuth2/OIDC for auth
- HTTP service-to-service calls for simplicity

1) Service naming + ports (baseline)

user-service `application.yml`

```
``yaml
spring:
  application:
    name: user-service
  server:
    port: 8080
``
```

order-service `application.yml`

```
``yaml
spring:
  application:
    name: order-service
  server:
    port: 8080
``
```

2) Service discovery

Option A (recommended on Kubernetes): Kubernetes DNS

****How it works****

- Service discovery is handled by K8s Service DNS.
- ``order-service`` calls ``http://user-service`` (Service name).

****K8s Service name****

- ``user-service.default.svc.cluster.local``

****No extra discovery client required**.**

Option B (non-K8s): Eureka/Consul

If not using K8s, you configure registration + discovery clients.

3) API Gateway

Typical responsibilities

- route ``/api/users/**`` → user-service
- route ``/api/orders/**`` → order-service
- auth at edge (optional but common)
- rate limits, request size limits

Spring Cloud Gateway (example)

****Gateway `application.yml`****

```
``yaml
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      httpclient:
        connect-timeout: 2000
        response-timeout: 5s
      routes:
        - id: user-service
          uri: http://user-service
      predicates:
        - Path=/api/users/**
      filters:
```

```
- StripPrefix=1
- id: order-service
uri: http://order-service
predicates:
- Path=/api/orders/**
filters:
- StripPrefix=1
```
```

### **\*\*Production notes\*\***

- Add rate limiting either in gateway or in WAF.
- Configure timeouts aggressively.

---

## **## 4) Centralized configuration**

### **### Option A: Kubernetes ConfigMap + env vars (common)**

- Keep app config in ConfigMaps
- Keep secrets in Secrets

Example `ConfigMap`:

```
```yaml
apiVersion: v1
kind: ConfigMap
metadata:
name: user-service-config
data:
APPLICATION_YML: |
app:
featureX: true
```
```

Mount as file or map to env vars.

### **### Option B: Spring Cloud Config / AWS SSM**

Use if you want centralized config with history and environment overlays.

---

## ## 5) Secrets management

### ### Common options

- AWS Secrets Manager (recommended on AWS)
- HashiCorp Vault
- Kubernetes Secrets

Example K8s Secret:

```
```yaml
apiVersion: v1
kind: Secret
metadata:
  name: user-service-secrets
type: Opaque
data:
  DB_PASSWORD: <base64>
```
```

---

## ## 6) Resilience (timeouts + circuit breaker + retry)

### ### Key rule

- Always configure **timeouts** first.
- Add **circuit breakers** per dependency.
- Use **retries** only for idempotent calls.

### ### Library (Resilience4j)

#### **Typical dependencies**

- `resilience4j-spring-boot3`
- `spring-boot-starter-actuator`

### ### Example: order-service calling user-service with circuit breaker

```
```java
@CircuitBreaker(name = "userService", fallbackMethod = "fallbackUser")
public UserDto getUser(String userId) {
    return restClient.get().uri("/api/v1/users/{id}", userId)
        .retrieve().body(UserDto.class);
}
```

```
private UserDto fallbackUser(String userId, Throwable ex) {
    throw new DependencyUnavailableException("user-service unavailable", ex);
}
```

```

### ### Production-style Resilience4j config (order-service)

```
```yaml
resilience4j:
  circuitbreaker:
    instances:
      userService:
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 50
        minimumNumberOfCalls: 20
        failureRateThreshold: 50
        slowCallRateThreshold: 50
        slowCallDurationThreshold: 2s
        waitDurationInOpenState: 30s
        permittedNumberOfCallsInHalfOpenState: 10
        automaticTransitionFromOpenToHalfOpenEnabled: true
        recordExceptions:
          - java.io.IOException
          - java.net.SocketTimeoutException
        timelimiter:
          instances:
            userService:
              timeoutDuration: 2s
```
```

---

## ## 7) Distributed tracing (OpenTelemetry)

### ### Goal

- Every request has `traceId` propagated across gateway → order-service → user-service.

### ### Typical production setup

- OpenTelemetry Java agent OR library instrumentation
- Trace backend: Jaeger/Tempo/X-Ray

### ### Spring Boot config (example)

```
```yaml
management:
tracing:
sampling:
probability: 0.1
```
```

#### **\*\*Production notes\*\***

- Don't sample 100% in high traffic.
- Ensure trace context propagation for outbound HTTP clients.

---

## ## 8) Centralized logging

### ### Requirements

- JSON logs
- include `traceId` in every log line

#### **\*\*Production notes\*\***

- Ship logs via FluentBit/Fluentd
- Store in ELK/Loki/CloudWatch

---

## ## 9) Metrics + monitoring + alerting

### ### Expose metrics

- Use Spring Boot Actuator + Micrometer
- Scrape via Prometheus or vendor agent

Example:

```
```yaml
management:
endpoints:
web:
exposure:
include: health,info,metrics,prometheus
```
```

Alerting best practice:

- Alert on SLO burn (error rate, latency), not raw CPU only.

---

## ## 10) Security (OAuth2/OIDC)

### ### Recommended model

- Gateway validates external tokens (optional)
- Services validate tokens (recommended) as resource servers

### \*\*Spring Boot resource server config (example)\*\*

```
``yaml
spring:
 security:
 oauth2:
 resourceserver:
 jwt:
 issuer-uri: https://<your-issuer>
``
```

### \*\*Production notes\*\*

- Use JWKS caching
- Apply least privilege (scopes/roles)

---

## ## 11) Service-to-service auth

Options:

- Internal JWT (service account tokens)
- mTLS via service mesh

Production note:

- If you adopt a mesh, mTLS is often the default and easiest to standardize.

---

## ## 12) Data layer



### ### Typical production requirements

- DB migrations (Flyway/Liquibase)
- connection pooling
- indexes aligned to queries
- backups and restore testing

Example Flyway config:

```
``yaml
spring:
 flyway:
 enabled: true
``
```

---

## ## 13) Messaging/eventing (optional but common)

If you have workflows (create order → billing → shipping), consider async events.

Options:

- Kafka
- SQS/SNS

Configuration considerations:

- DLQ
- retries
- idempotent consumers
- schema versioning (Schema Registry)

---

## ## 14) Kubernetes deployment (user-service and order-service)

### ### Deployment (3 replicas)

```
``yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: user-service
```

```
spec:
 replicas: 3
 selector:
 matchLabels:
 app: user-service
 template:
 metadata:
 labels:
 app: user-service
 spec:
 containers:
 - name: user-service
 image: <registry>/user-service:1.0.0
 ports:
 - containerPort: 8080
 resources:
 requests:
 cpu: 100m
 memory: 256Mi
 limits:
 cpu: 500m
 memory: 512Mi
 readinessProbe:
 httpGet:
 path: /actuator/health
 port: 8080
 initialDelaySeconds: 10
 periodSeconds: 10
 livenessProbe:
 httpGet:
 path: /actuator/health
 port: 8080
 initialDelaySeconds: 20
 periodSeconds: 10
 ...
```

### ### Service

```
```yaml
apiVersion: v1
kind: Service
metadata:
```

```
name: user-service
spec:
  selector:
    app: user-service
  ports:
    - port: 80
    targetPort: 8080
````
```

Repeat similarly for `order-service`.

---

## ## 15) Autoscaling (HPA)

```
``yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: user-service-hpa
spec:
 scaleTargetRef:
 apiVersion: apps/v1
 kind: Deployment
 name: user-service
 minReplicas: 3
 maxReplicas: 10
 metrics:
 - type: Resource
 resource:
 name: cpu
 target:
 type: Utilization
 averageUtilization: 70
````
```

16) CI/CD

Typical pipeline stages:

- build + unit tests
- integration tests
- security scans (SCA + container scan)
- build image + push to registry
- deploy (ArgoCD/Flux or pipeline apply)

Production notes:

- use canary/blue-green for risky changes
- always have rollback

17) Container image registry

You need:

- ECR/GCR/ACR/DockerHub
- image tags (git SHA)
- vulnerability scanning

18) Service mesh (optional)

Use if you need:

- standardized mTLS
- traffic shifting
- consistent telemetry

Options:

- Istio
- Linkerd

19) Edge protection (WAF + rate limiting)

On AWS:

- AWS WAF in front of ALB/API Gateway
- throttle abusive clients

20) Governance / platform standards

Examples:

- standard API error format
- standard logging/tracing libs
- standard Helm charts
- dependency update and CVE policy

“Any other tools/services required?” (common production add-ons)

- Registry (ECR)
- Secrets manager (Secrets Manager/Vault)
- Observability backend (Prometheus/Grafana + Jaeger/Tempo + log store)
- CI/CD (GitHub Actions/Jenkins + ArgoCD)
- Messaging broker (Kafka/SQS) and possibly schema registry
- Optional service mesh

Microservices Tooling & Configuration?

Spring Boot 3.x / Spring Security 6 / Spring Cloud (Current Release Train)

Production-oriented reference for tools, concepts, configurations, and annotations used in modern Java microservices.

Note

“Spring Boot 4” is treated as a forward-looking label. Architecture and configurations remain valid for Spring Boot 3.x.

Table of Contents

1. Core Concepts
2. Service Discovery
3. API Gateway
4. Resilience
5. Observability
6. Messaging & Eventing
7. Security Baseline
8. Deployment & Configuration Management
9. Spring Annotations Cheat Sheet
10. Interview Mapping Guide

1. Core Concepts

Service Design

- Service boundaries (DDD): service owns its data & lifecycle
- Database-per-service
- Async-first communication where appropriate
- Idempotent APIs

Data & Consistency Patterns

- Saga + compensation
- Transactional outbox
- Retry + deduplication
- Backpressure + DLQ

Reliability & Operations

- SLO / SLI / Error budgets
- Observability = Logs + Metrics + Traces
- Safe retries and timeout policies

2. Service Discovery

Modern Recommendation

| Environment | Preferred Approach |
|--------------------------|------------------------|
| Kubernetes (EKS/GKE/AKS) | Kubernetes Service DNS |
| Cloud VM / Hybrid | Cloud Map / Managed LB |
| Legacy Spring Cloud | Eureka |

Eureka is commonly asked in interviews but rarely needed inside Kubernetes.

Eureka Server (Spring Boot 3)

Dependency (concept)

- Spring Cloud BOM
- Eureka server starter

Main Class

```
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(DiscoveryServerApplication.class, args);
    }
}
```

application.yml

```
server:
  port: 8761

spring:
  application:
    name: discovery-server

eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

Eureka Client

application.yml

```
spring:
  application:
    name: order-service

eureka:
  client:
    serviceUrl:
```

defaultZone: <http://localhost:8761/eureka/>

Load Balanced RestTemplate

```
@Bean
@LoadBalanced
RestTemplate restTemplate() {
    return new RestTemplate();
}
```

3. API Gateway (Spring Cloud Gateway)

Responsibilities

- Routing
- Authentication & authorization
- Rate limiting
- Request normalization

Avoid business logic in gateway.

Route Configuration

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: http://localhost:8081
          predicates:
            - Path=/orders/**
          filters:
```

- StripPrefix=0

Rate Limiting (Redis Token Bucket)

filters:

- name: RequestRateLimiter

args:

redis-rate-limiter.replenishRate: 10

redis-rate-limiter.burstCapacity: 20

key-resolver: "#{@ipKeyResolver}"

@Bean

```
KeyResolver ipKeyResolver() {  
    return exchange -> Mono.just(  
        exchange.getRequest().getRemoteAddress().getAddress().getHostAddress()  
    );  
}
```

4. Resilience (Resilience4j)

Concepts

- Timeouts at every hop
- Bounded retries + jitter
- Circuit breakers
- Bulkheads

Configuration

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        slidingWindowSize: 20
        failureRateThreshold: 50
        waitDurationInOpenState: 10s

  retry:
    instances:
      paymentService:
        maxAttempts: 3
        waitDuration: 200ms
```

Usage

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
@Retry(name = "paymentService")
public String charge(String orderId) {
    return "OK";
}

private String fallback(String orderId, Throwable ex) {
    return "PENDING";
}
```

5. Observability

Tracing (OpenTelemetry)

management:

tracing:

sampling:

probability: 1.0

otlp:

tracing:

endpoint: <http://localhost:4318/v1/traces>

Propagate context across:

- HTTP calls
- Kafka messages

Metrics (Micrometer + Prometheus)

management:

endpoints:

web:

exposure:

include: health,info,metrics,prometheus

Track:

- RED metrics (Rate, Errors, Duration)
- JVM metrics
- DB & downstream latency

Logging

- Structured JSON logs
- Include correlation ID + trace ID
- Avoid PII

Backends: ELK / Loki / CloudWatch / OpenSearch

6. Messaging & Eventing

Kafka Concepts

- At-least-once delivery
- Idempotent consumers
- Retry topics or DLQ
- Partition ordering

Consumer Example

```
@KafkaListener(topics = "order-events", groupId = "order-consumer")
public void onMessage(String payload, Acknowledgment ack) {
    // validate
    // process
    ack.acknowledge();
}
```

```
spring:
  kafka:
    consumer:
      enable-auto-commit: false
    listener:
```

ack-mode: manual

AWS Messaging Alternatives

| Service | Use Case |
|-------------|---------------------|
| SQS | Queue processing |
| SNS | Pub-sub fanout |
| EventBridge | Event routing |
| Kinesis | Streaming analytics |

7. Security Baseline

- Validate JWT at gateway and/or service
- Method-level authorization
- Avoid logging sensitive data
- Optional mTLS via service mesh

8. Deployment & Configuration Management

Deployment

- Docker containers
- Kubernetes manifests
- Canary releases

Configuration

- Externalized config

- Feature flags
- Safe defaults

Tools

- Spring Cloud Config
- AWS AppConfig
- LaunchDarkly / Unleash

9. Spring Annotations Cheat Sheet

Core

@SpringBootApplication — application entry
@Configuration — config class
@Bean — bean definition
@Component / @Service / @Repository — stereotypes
@ConfigurationProperties — bind config

Web

@RestController
@GetMapping / @PostMapping
@Valid / @Validated

Discovery

@EnableEurekaServer
@EnableDiscoveryClient
@LoadBalanced

Resilience4j

@CircuitBreaker

@Retry

@Bulkhead

@TimeLimiter

Messaging

@KafkaListener

Async & Scheduling

@Async

@Scheduled

10. Interview Mapping Guide

This document covers tooling and configuration.

Use your master interview question bank for scenario-based Q&A.

Map topics as follows:

| Interview Topic | Section |
|-----------------------|---------------------|
| Service communication | Discovery + Gateway |
| Failure handling | Resilience |
| Monitoring | Observability |
| Event-driven systems | Messaging |

| | |
|----------------------|---------------------|
| Security | Security baseline |
| Platform engineering | Deployment & config |

Microservices Interview Questions

Beginner → Expert Reference Guide

1. Fundamentals & Architecture

Basics

- What is microservices architecture?
- Why microservices over monolith? Tradeoffs?
- What is a modular monolith? When is it better?
- What is a distributed monolith? How do you detect it?

Design & Boundaries

- How do you decide service boundaries?
- Applying DDD bounded contexts in microservices
- How Conway's Law affects architecture

- Common microservices anti-patterns

2. API Design & Contracts

API Principles

- Designing REST APIs for microservices
- REST vs gRPC — when and why?
- API pagination, filtering, sorting design
- Designing consistent error responses
- HATEOAS — when to use?

Versioning & Compatibility

- API versioning strategies
- Backward compatibility
- Handling breaking changes
- Consumer-driven contract testing

Idempotency

- Designing idempotent APIs
- Safe retries for POST endpoints

3. Communication Patterns

- Sync vs async communication
- Request/response vs event-driven
- Event-driven architecture basics
- Pub/Sub vs Message Queue
- Preventing chatty communication
- Outbox pattern
- Event schema evolution

4. Data & Transactions

Data Ownership

- Why database-per-service?
- Is shared database ever acceptable?

Distributed Transactions

- Handling transactions without 2PC
- Saga pattern
- Choreography vs orchestration
- Compensation actions

Advanced Patterns

- CQRS usage
- Event sourcing (when NOT to use)
- Reporting across services
- Source of truth & duplication

5. Consistency, Ordering & Idempotency

- Eventual consistency — where acceptable?
- Delivery semantics: at-most / at-least / exactly once
- Handling duplicate messages
- Out-of-order events
- Kafka ordering strategy
- DLQ usage
- Poison message handling
- Retry storm prevention

6. Resilience & Fault Tolerance

- Circuit breaker
- Bulkhead isolation
- Rate limiting placement
- Timeout strategies
- Retry policies
- Graceful degradation
- Backpressure

7. Service Discovery & Routing

- Service discovery concepts
- Client-side vs server-side discovery
- API Gateway responsibilities
- Gateway vs BFF
- API aggregation/composition
- Cross-cutting concerns (auth/logging/throttling)

8. Security

- OAuth2 vs OIDC
- JWT vs opaque tokens
- Service-to-service authentication
- mTLS usage
- Secrets management
- RBAC vs ABAC authorization
- Securing internal endpoints
- Multi-tenant isolation & data leakage prevention

9. Observability

- Observability vs monitoring
- Logs vs metrics vs traces
- Correlation ID propagation
- Distributed tracing with OpenTelemetry
- SLIs / SLOs / SLAs
- RED vs USE methods
- Alert noise reduction
- Debugging multi-service production issues

10. Deployment & Release Engineering

- Blue-green vs canary vs rolling
- Feature flags usage
- Safe rollback strategies
- Zero-downtime DB migration
- Environment config management
- GitOps workflow
- Secret handling in CI/CD

11. Kubernetes & Platform

- Pod / Deployment / Service / Ingress
- Readiness vs Liveness vs Startup probes
- Resource requests vs limits
- HPA limitations
- Debugging CrashLoopBackOff
- Handling node failures
- Network policies importance

12. Service Mesh

- Problems service mesh solves
- When mesh is unnecessary
- Sidecar overhead
- Retry/timeout in mesh vs app layer

13. Performance & Scalability

- Reducing p95/p99 latency
- Tail latency
- Caching layers (CDN/client/app/DB)
- Cache strategies comparison
- Cache invalidation
- Load testing & capacity planning

14. Testing Strategy

- Unit vs Integration vs Contract vs E2E
- Testing async flows
- Testing Sagas
- Test data management
- Testcontainers usage
- Avoiding flaky tests

15. Multi-Region & Disaster Recovery

- Multi-AZ vs multi-region
- Active-active vs active-passive
- RPO vs RTO

- Chaos engineering & failover testing

System Design Prompts

- Order management system (inventory + payment + shipping)
- Payment system with reconciliation & idempotency
- Event-driven notification system with retries & DLQ
- Public API platform with rate limiting & quotas
- Multi-tenant SaaS backend
- Real-time analytics pipeline (Kafka streaming)

Practical Implementation Questions

REST & Spring Boot

- Validation + pagination + error schema
- Global exception handling
- Idempotent POST endpoint
- Correlation ID propagation
- OpenAPI schema enforcement

Resilience

- Timeouts + retries
- Circuit breaker fallback
- Bulkhead isolation
- Per-user rate limiting

Messaging

- Kafka producer with versioning
- Idempotent consumer
- Retry + DLQ handling
- Deduplication store (Redis/DB)
- Partition ordering strategy
- Outbox + idempotent consumer processing

Saga Patterns

- Orchestration state machine
- Compensation handling
- Choreography via events

Data & CQRS

- Separate write/read models
- Event replay & projection rebuild
- Optimistic locking

Security

- OAuth2 resource server
- RBAC implementation
- Service-to-service authentication
- Secure secrets loading

Observability

- Structured logs
- Custom metrics
- Distributed tracing instrumentation
- Immutable audit logs

Kubernetes Readiness

- Proper readiness/liveness endpoints
- Graceful shutdown handling
- Dependency-aware health checks

Performance

- Cache-aside implementation
- High throughput endpoint tuning
- Async processing

Testing

- Contract tests
- Testcontainers integration tests
- Idempotency tests
- Saga compensation tests

Outdated Topics (Mention Carefully)

- SOA vs Microservices as primary comparison
- ESB-centric architecture
- Netflix OSS default stack (Zuul/Hystrix/Eureka everywhere)